

cōdearti

Curso

ARQUITECTURA DE MICROSERVICIOS CON SPRING BOOT Y KUBERNETES

Spring Boot, WebFlux, Docker, Open API, Kafka, Prometheus, Elastic Stack, Jaeger, K8s,...

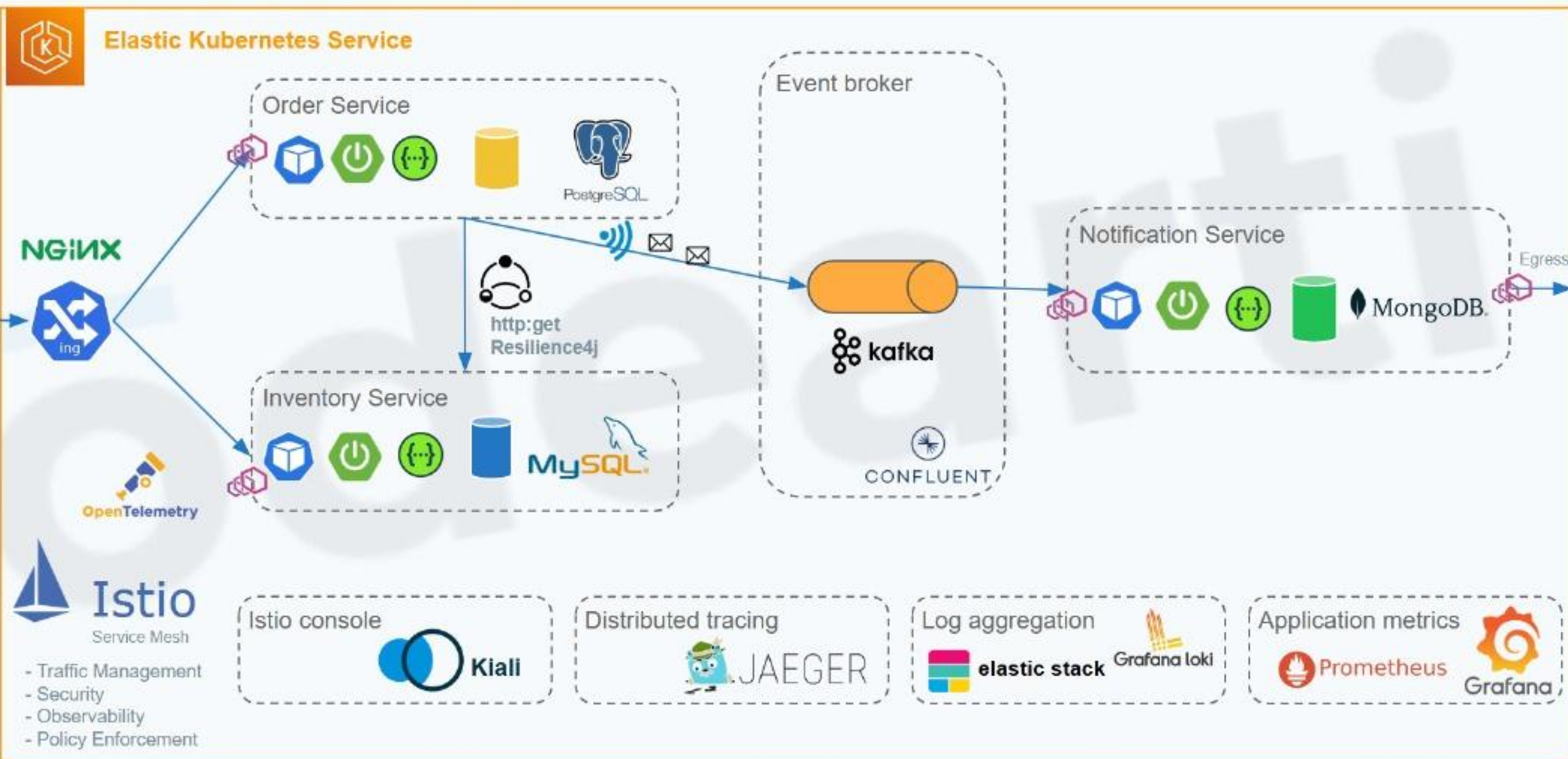
KUBERNETES (K8S)



CURSO DE ARQUITECTURA DE MICROSERVICIOS CON SPRING BOOT Y KUBERNETES



- Deployment
- Service
- Secrets & Config Maps
- Horizontal Pod Autoscaler
- Registry & Discovery
- Load Balancer



¿Qué aprenderemos en esta sesión 3?

Arquitectura Event Driven y Patrones de Observabilidad:

- Event Driven Architecture
- Confluent y Apache Kafka
- Patrones de Observability
 - Log Aggregation
 - Application Metrics
 - Health Check Api
 - Exception Tracking
 - Distributed Tracing and Monitoring
- Construcción del servicio de notificaciones (notifications-service)
- Despliegue de base de datos sobre docker (MongoDB)
- Enviar notificación en el servicio de pedido (order service)
- Despliegue del event broker sobre docker (Kafka)
- Despliegue sobre docker de jeager, elastic stack, prometheus y grafana

Sesión 3



Event Driven Architecture

Servicio de E-Commerce

Comunicación Entre Microservicios



- El servicio 1 depende directamente del servicio 2.
- Tiene que esperar por la respuesta de forma síncrona (o añadir lógica a mayores)

Event Driven Architecture

Patrón de arquitectura software en el que el flujo de información entre los distintos subsistemas viene determinado por **eventos**.



Aspectos Positivos

- Desacople entre las partes publicadoras y consumidoras del evento.
- Flexibilidad, es muy sencillo añadir o eliminar consumidores sin modificar el productor.
- Proceso asíncrono sencillo, sin la necesidad de gestionar hilos.
- Al estar las partes más desacopladas, puede ser más sencillo de testear.

Aspectos Negativos

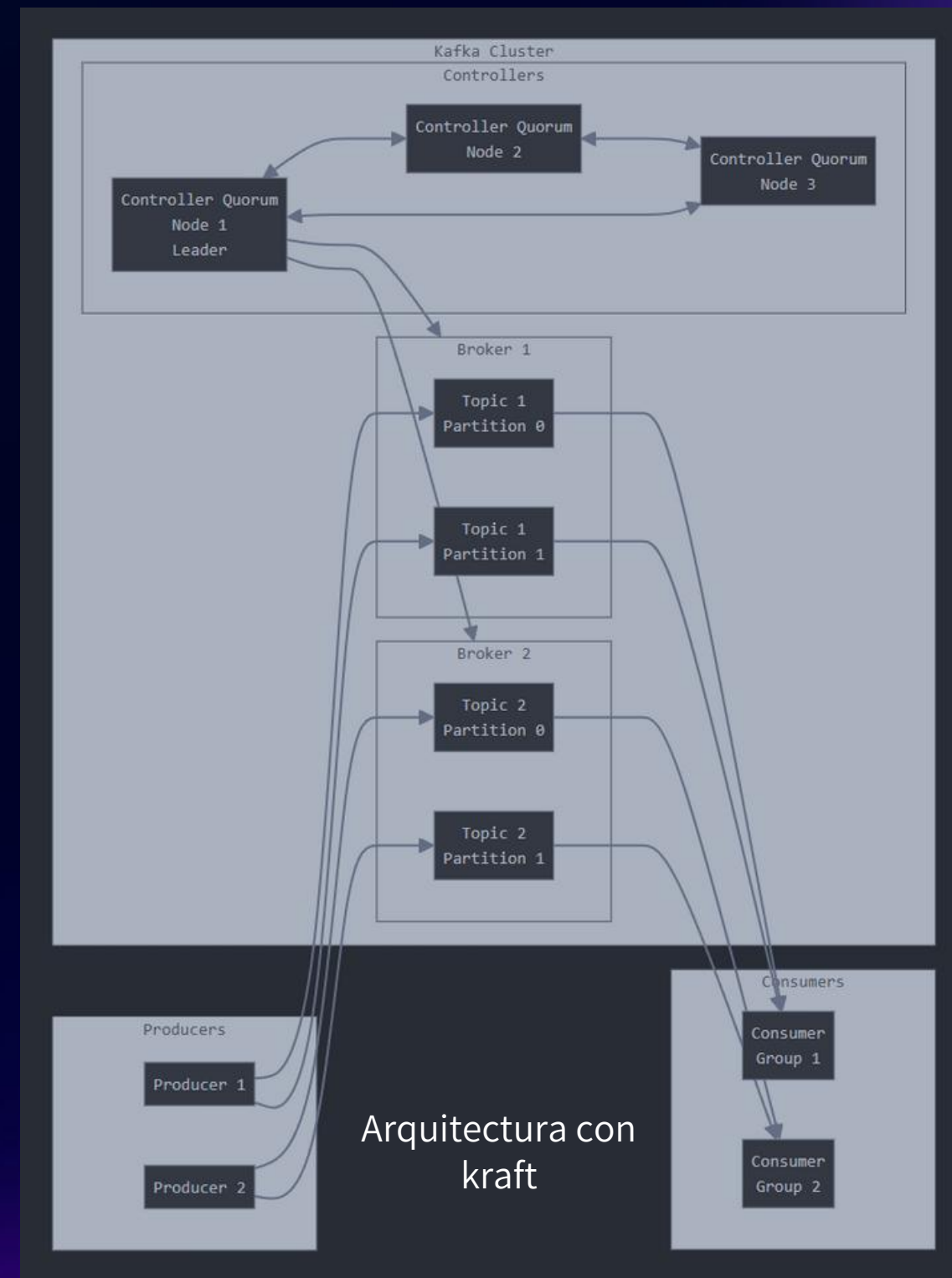
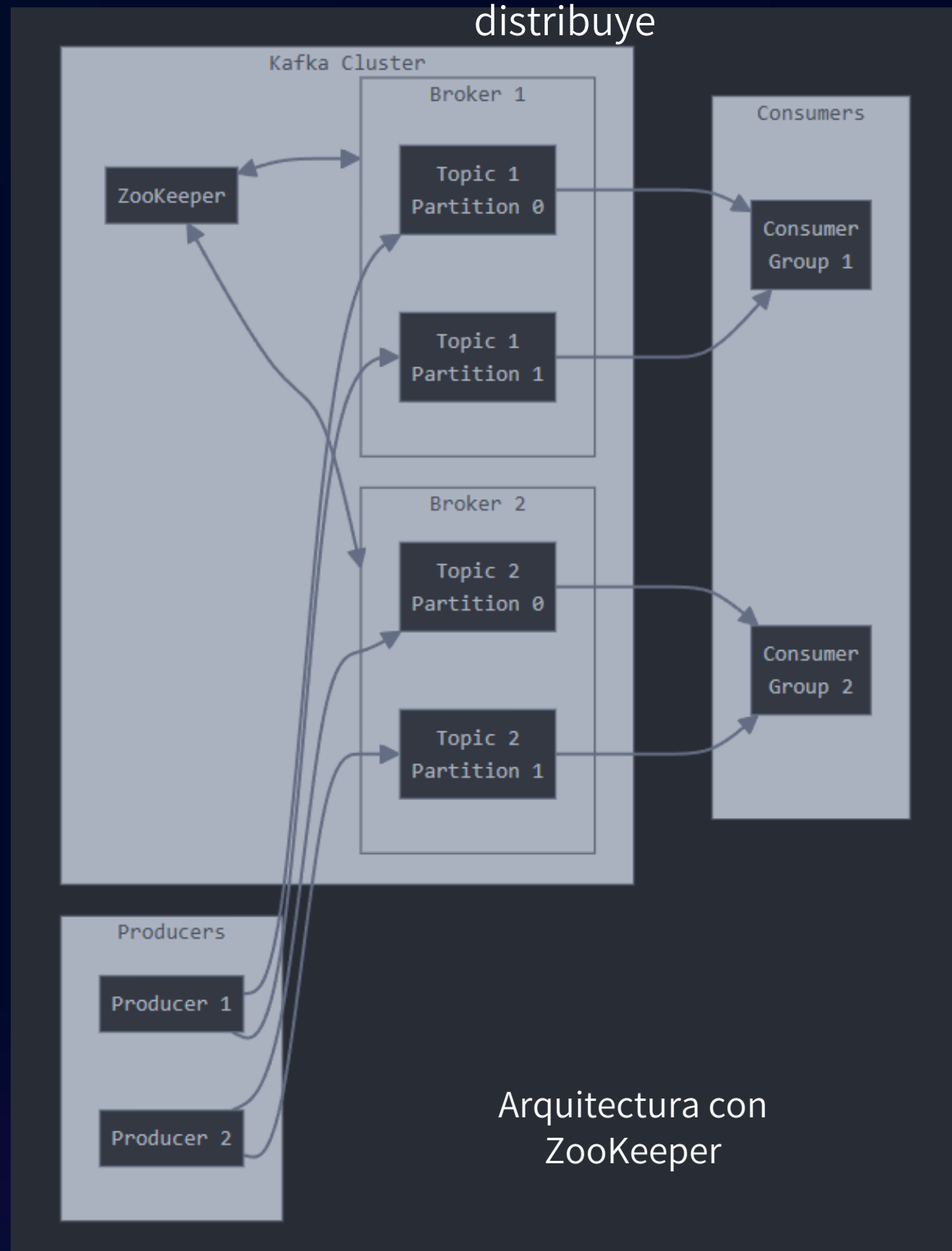
- Es más complicado seguir el flujo.
- Tarea de debug más difícil.
- Añade la complejidad del Event Publisher.
 - Mitigado usando frameworks como .NET o Spring.

¿Cuándo Usar?

- Cuando en el futuro probablemente necesitemos más entidades que procesen el mismo evento.
- Necesidad de operaciones asíncronas.
- Cuando la parte productora no necesite el resultado de la parte que escucha y procesa el evento.

Apache Kafka

Sistema central que recibe eventos y los distribuye



- Reemplaza a ZooKeeper para la gestión de metadata.
- Los nodos pueden ser dedicados o combinados con brokers.
- Simplifica la arquitectura (un componente menos).
- Mejor rendimiento en operaciones de metadata.
- Menor latencia en recuperación de fallos.
- Mejor escalabilidad.
- Más fácil de operar y mantener.

Apache Kafka

Topics

- Divididos en particiones para paralelismo
- Cada partición es una secuencia ordenada de mensajes

Producers

- Publican mensajes a los topics
- Pueden elegir a qué partición enviar los mensajes
- Soportan distribución de carga y tolerancia a fallos

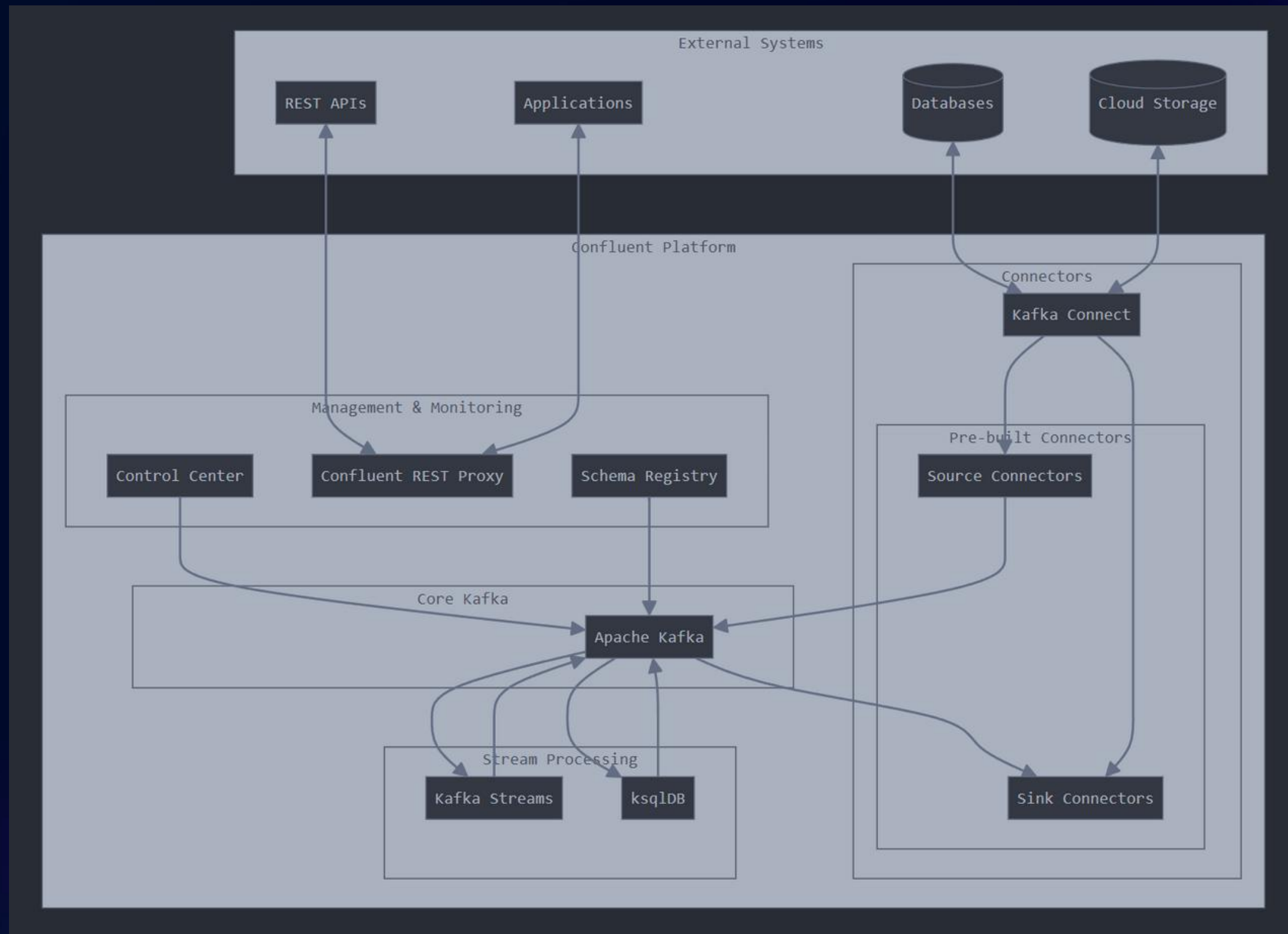
Brokers

- Servidores que forman el cluster de Kafka
- Almacenan las particiones de los topics
- Manejan las peticiones de lectura/escritura

Consumers

- Leen mensajes de los topics
- Se organizan en consumer groups para procesamiento paralelo
- Cada partición es leída por un solo consumer del grupo

Confluent



Plataforma de streaming basada en Apache Kafka cuyo objetivo es ayudar a los diferentes roles en la organización en los proyectos de intercambio de información en tiempo real.

- Capacidades de Streaming Enterprise
- Conectividad y Gobierno del Dato
- Procesamiento Avanzado
- Monitorización y Gestión
- Seguridad
- Arquitecturas MultiSite y Apoyo en la evolución hacia el Cloud

Confluent

Confluent Control Center

- Dashboard central de monitoreo
- Gestión de cluster y topics
- Monitoreo de producción/consumo
- Alertas y métricas
- Debugging de streams

Schema Registry

- Gestión centralizada de schemas
- Validación de compatibilidad
- Evolución controlada de schemas
- Soporte para Avro, Protobuf, JSON Schema

ksqlDB

- Base de datos para streaming
- Queries SQL para procesamiento de streams
- Creación de aplicaciones event-driven
- Procesamiento en tiempo real

Kafka Connect

- Framework de integración de datos
- Conectores pre-construidos:
 - Sources: MySQL, PostgreSQL, MongoDB, etc.
 - Sinks: S3, Elasticsearch, HDFS, etc.
- Escalable y tolerante a fallos
- Transformaciones de datos

REST Proxy

- API REST para Kafka
- Independiente del lenguaje
- Producir/consumir mensajes vía HTTP
- Gestión de topics y schemas

Confluent - Ventajas - Casos de Uso

Ventajas de Confluent

- Gestión simplificada
- Herramientas empresariales
- Soporte profesional
- Seguridad mejorada
- Multi-cloud y hybrid-cloud
- Monitoreo avanzado

The logo for Confluent Platform consists of a white square with the text "Confluent Platform" inside. The square is framed by a white border that has a stepped, L-shaped appearance on the top and right sides.

Confluent
Platform

Casos de uso comunes

- ETL en tiempo real
- Análisis de datos en streaming
- Procesamiento de eventos
- Integración de microservicios
- CDC (Change Data Capture)

The logo for Confluent Cloud consists of a white square with the text "Confluent Cloud" inside. The square is framed by a white border that has a stepped, L-shaped appearance on the top and right sides.

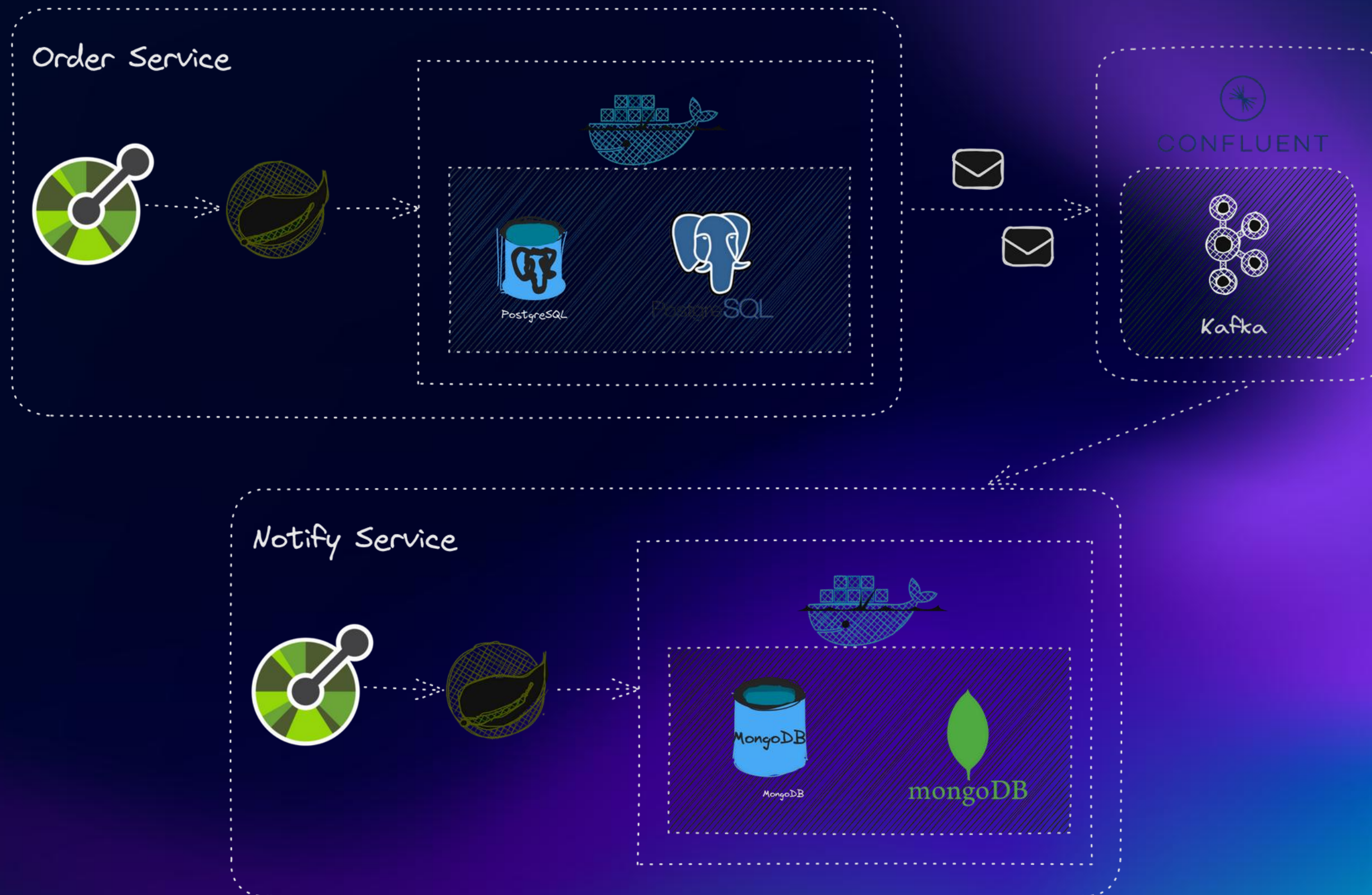
Confluent
Cloud



Implementación

Servicio de E-Commerce

API - Servicio de Notificación



Sesión 3

03

Patrones de Observabilidad

Servicio de E-Commerce

Log Aggregation

Permite centralizar y procesar los logs generados por cada servicio para obtener una visión integral de todo el sistema.

Facilita:

- Consultas y filtrado
- Correlación de Eventos entre servicios
- Detección de patrones de error



Log Aggregation

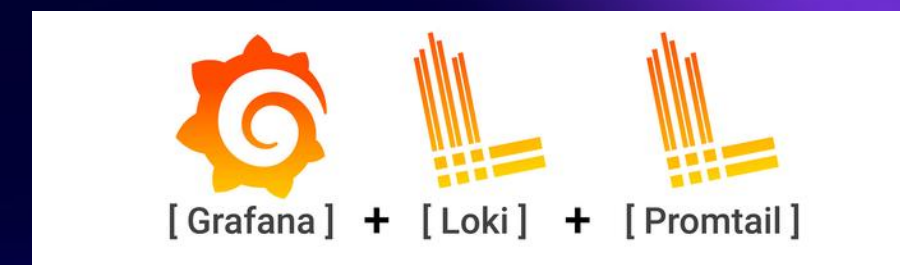
Elastic Stack (ELK)

Incluye Elasticsearch, Logstash, y Kibana.
Elasticsearch almacena los logs, Logstash los procesa y filtra, y Kibana permite visualización y análisis.



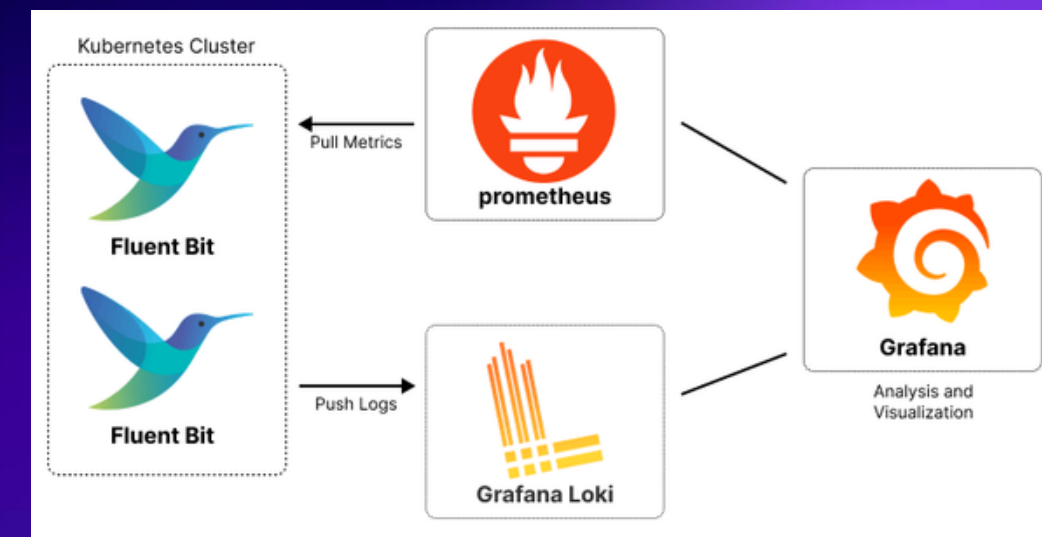
Grafana Loki

Un sistema de agregación de logs diseñado para integrarse con Grafana y consumir menos recursos que ELK, ideal para configuraciones en Kubernetes.



Fluentd

Es una herramienta de recolección de logs altamente escalable, con soporte para varios formatos y sistemas de almacenamiento. Se integra bien con sistemas de monitoreo como Prometheus y Loki.



Application metrics

Este patrón se centra en recolectar y analizar métricas de los servicios y aplicaciones que ayudan a comprender el comportamiento de la aplicación, diagnosticar problemas y mejorar el rendimiento general del sistema.

- Métricas de rendimiento
- Métricas de capacidad
- Métricas de negocio
- Métricas de tráfico

Instrumentación del
Código



Recolección de
Métricas



Visualización de
Métricas



Application metrics

Importancia

- Diagnosticar problemas rápidamente
- Detectar patrones y anomalías
- Optimizar el rendimiento
- Escalar de manera proactiva

Tipos de Métricas

- Contadores (Counters)
- Medidas de Temporización (Timers)
- Medidores (Gauges)
- Histograms y Percentiles

Buenas Prácticas

- Usa métricas normalizadas
- Evita generar métricas en exceso
- Automatiza alertas proactivas

Health Check Api

Permite monitorear el estado y la salud de nuestros microservicios

Tipos de Health Checks

Liveness Check

- Verifica si el servicio está "vivo" y ejecutándose
- Responde si la aplicación está activa o no
- Útil para que Kubernetes u otros orquestadores sepan si deben reiniciar el contenedor

Readiness Check

- Indica si el servicio está listo para recibir tráfico
- Verifica dependencias críticas (bases de datos, caches, otros servicios)
- Ayuda a prevenir el envío de tráfico a instancias no preparadas

Permite a otros sistemas (monitores, balanceadores, orquestadores) verificar si el servicio está funcionando correctamente



Exception Tracking

Exception Global - @ControllerAdvice
Exception Especifico - @ExceptionHandler

- Permite capturar, registrar y rastrear excepciones de manera centralizada.
- Facilita el análisis y diagnóstico de problemas en producción.
- Ayuda a mantener la calidad y estabilidad del sistema.

- Categorización de Excepciones
 - Clasificar por severidad
 - Agrupar por tipo
 - Identificar patrones comunes
- Retención de Datos
 - Definir políticas de retención
 - Archivar datos históricos
 - Mantener índices eficientes
- Seguridad
 - Sanitizar datos sensibles
 - Implementar control de acceso
 - Encriptar información crítica
 - Ejemplo de Configuración

Distributed Tracing and Monitoring

- Permite rastrear y monitorear el flujo de una petición a través de múltiples microservicios
- Facilita la identificación de cuellos de botella y problemas de rendimiento
- Proporciona visibilidad end-to-end de las transacciones

Los principales headers utilizados son:

- X-B3-TraceId:
 - Identificador único que representa toda la transacción end-to-end.
 - Este ID se propaga a través de todos los servicios involucrados.
- X-B3-SpanId:
 - Identificador único de cada segmento o span de la traza.
 - Cada servicio que procesa la petición genera su propio span con su ID.
- X-B3-ParentSpanId:
 - Identifica el span padre en la jerarquía.
 - Esto permite reconstruir la estructura de la traza.
- X-B3-Sampled:
 - Flag que indica si la traza debe ser muestreada y registrada.
 - Esto permite controlar el volumen de trazas recolectadas.

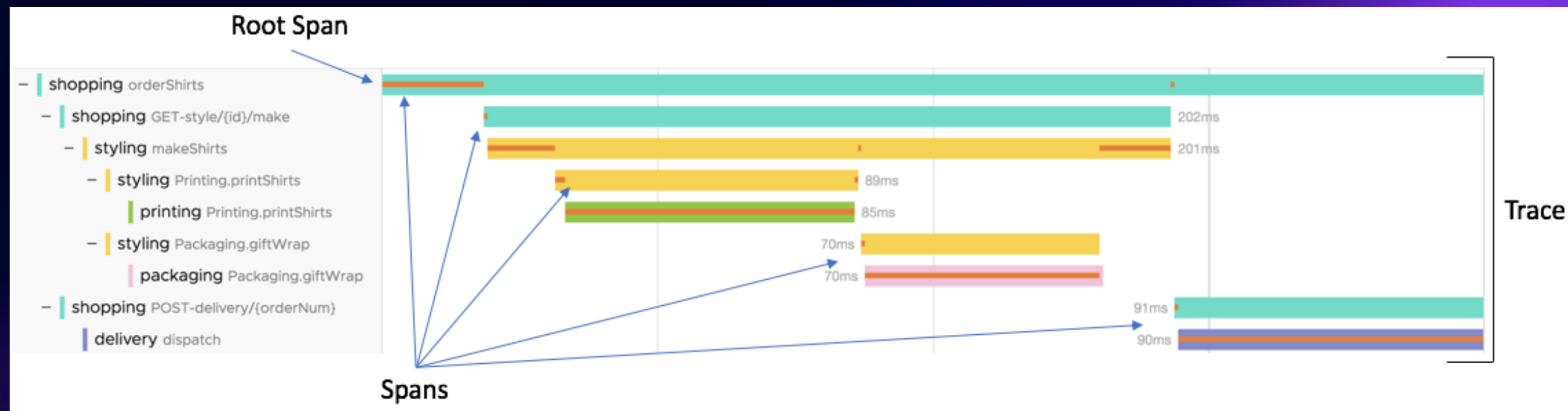


Distributed Tracing and Monitoring

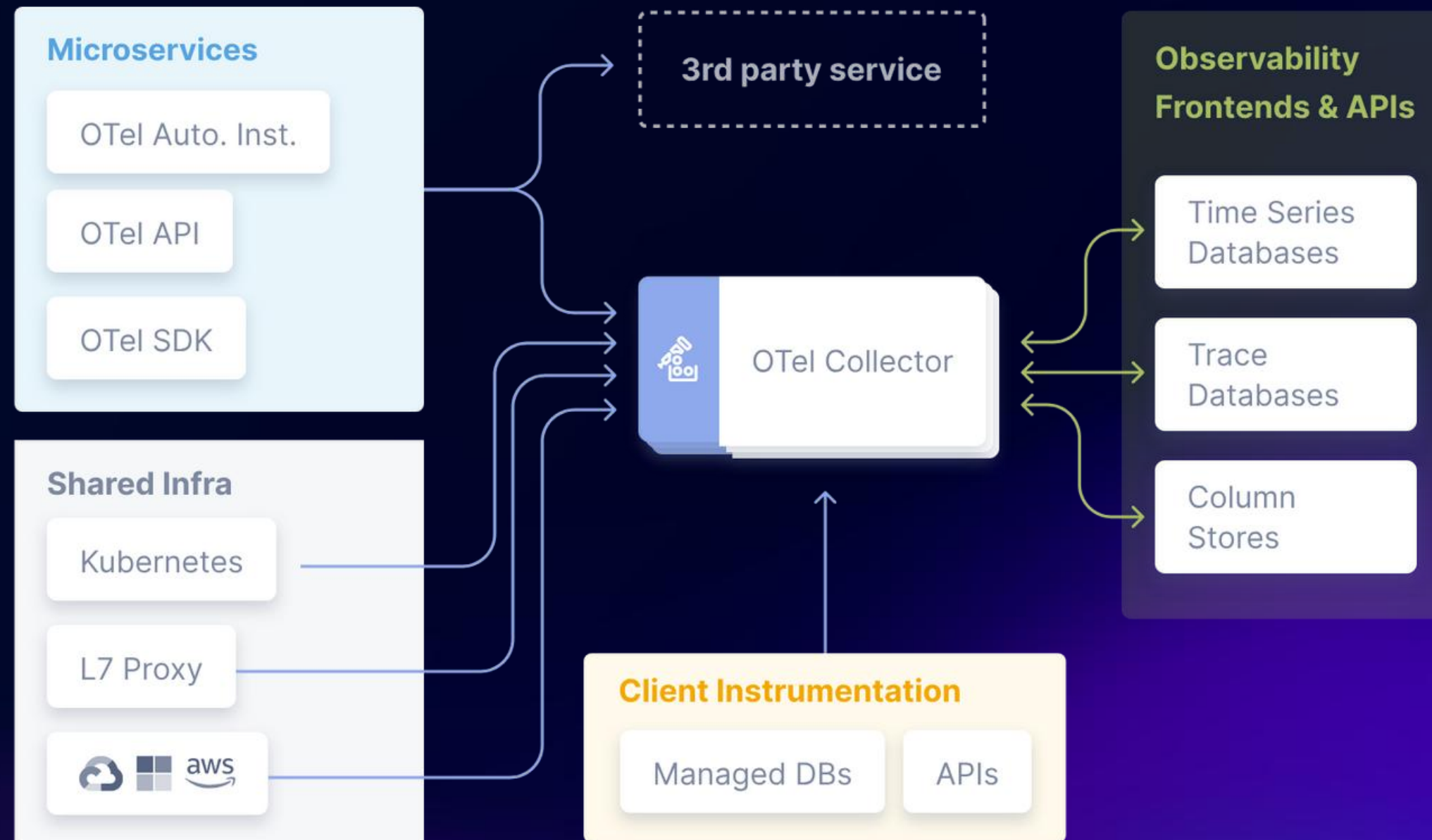


traceparent: 00-{TraceId}-{SpanId}-{TraceFlags}

- TraceId:
 - Identificador único de 32 caracteres hexadecimal que representa toda la transacción.
- SpanId:
 - Identificador único de 16 caracteres hexadecimal que representa el span actual.
- TraceFlags:
 - Un byte en hexadecimal que indica opciones de la traza, como si está siendo muestreada.



Open Telemetry



Traces



Metrics



Logs

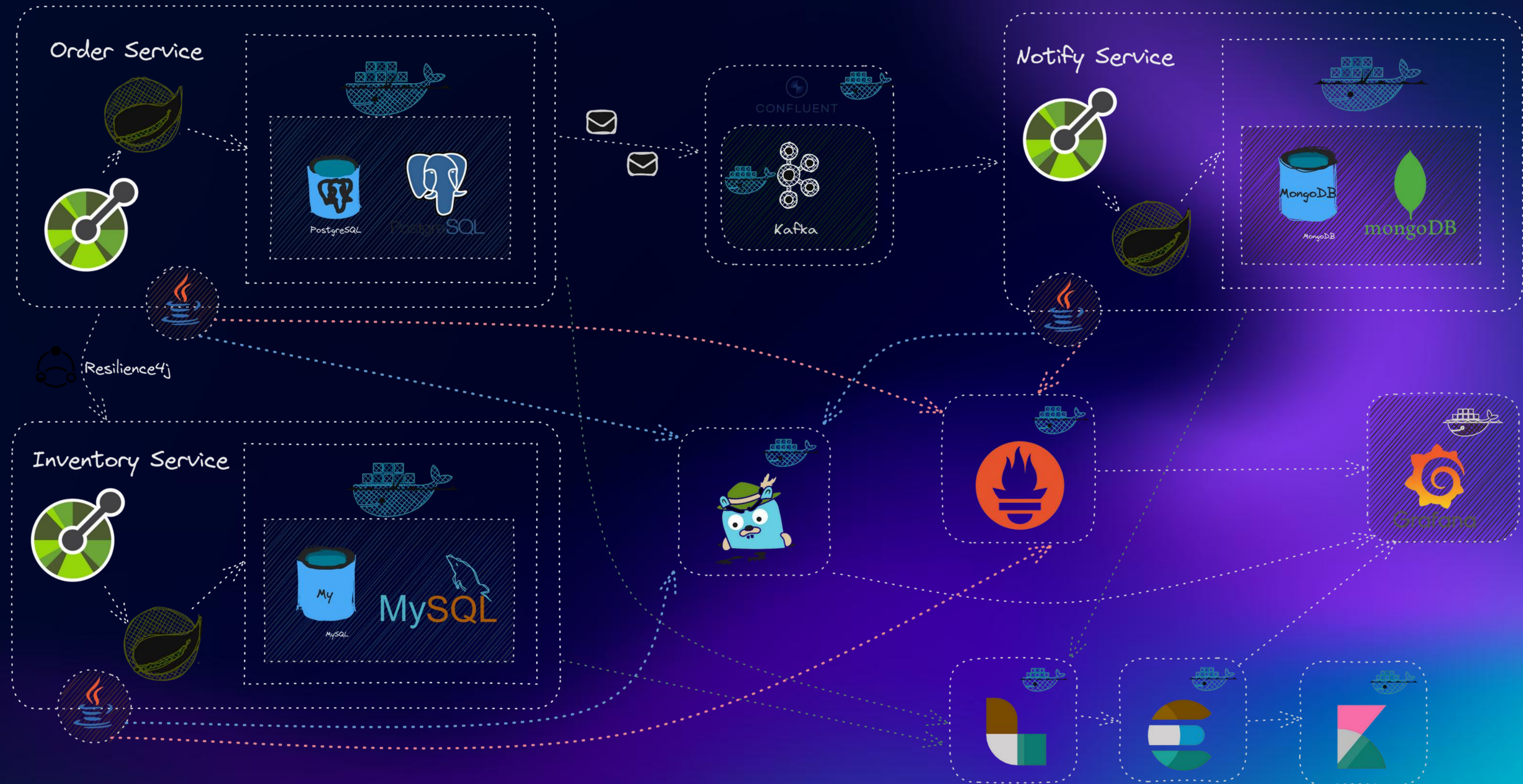




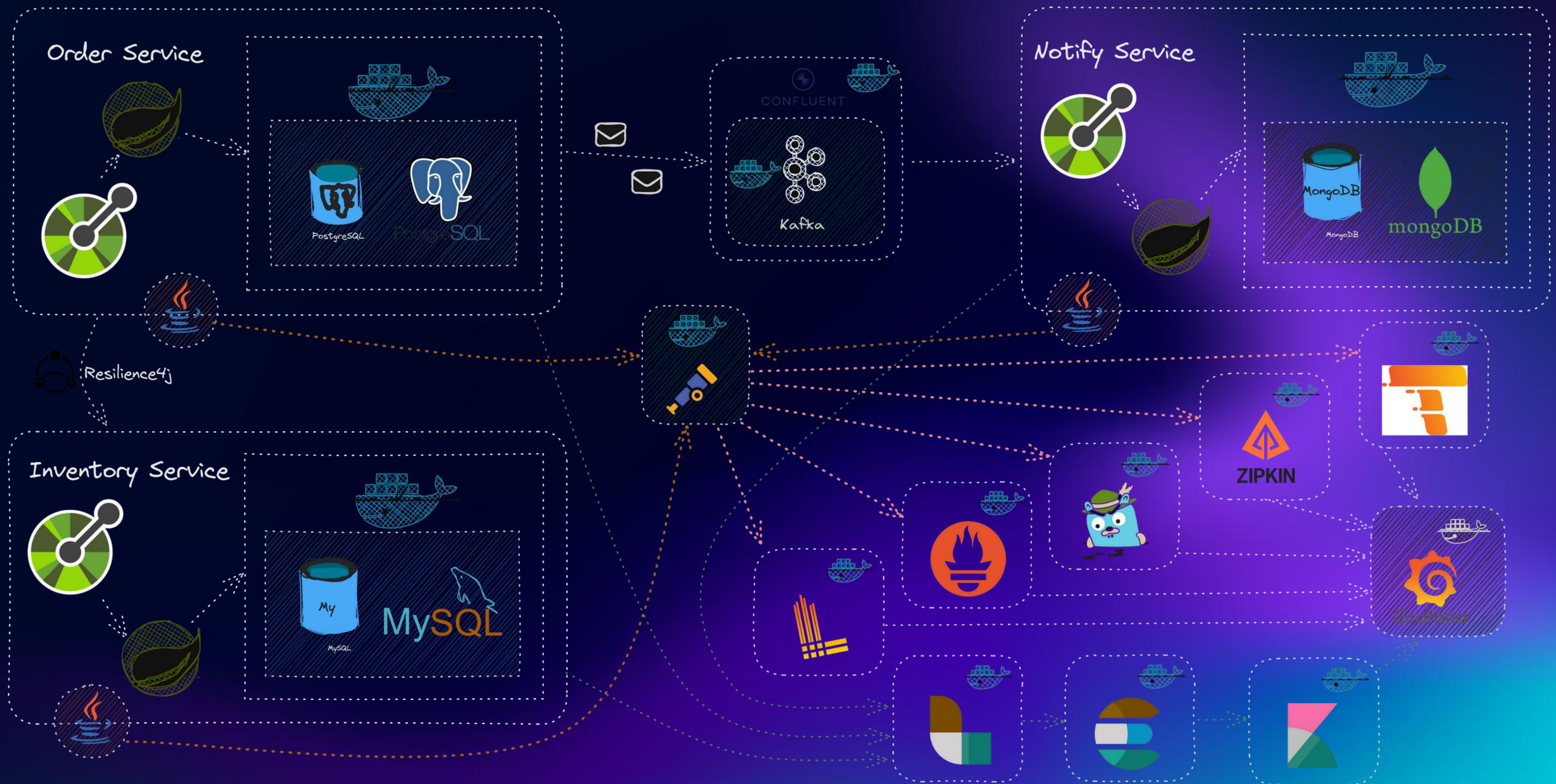
Implementación

Servicio de E-Commerce

Observabilidad en los Servicios

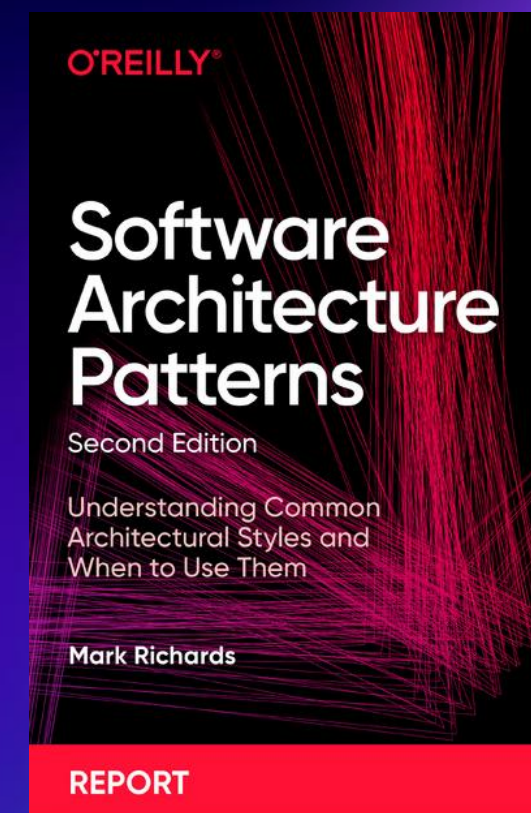
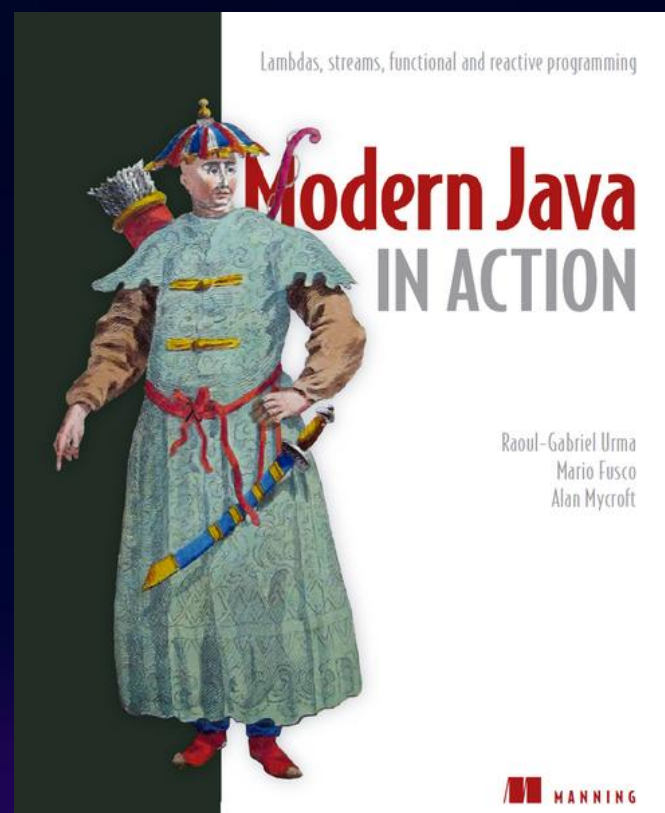


Observabilidad en los Servicios



Referencias

- <https://opentelemetry.io/docs/>
- <https://www.jaegertracing.io/docs/1.61/getting-started/>
- <https://zipkin.io/pages/quickstart.html>
- <https://grafana.com/docs/grafana/latest/>



Gracias!



¿Tienes alguna Pregunta?

contacto@codearti.com

+51 953 888 029

codearti.com